



COMSATS University Islamabad, Attock campus
Department of Computer Science

Program: BS-SE

Sp24-BSE-058

Sumbal

Lab mid term

Information security

Ma'am Ambreen GUL

Question # 01:

Write a Python program that performs both encryption and decryption using XOR operation.

```
# XOR Encryption and Decryption
message = input("Enter message: ")
key = input("Enter single-character key: ")

# Encryption
ciphertext = ''
for char in message:
    ciphertext += chr(ord(char) ^ ord(key))

# Decryption (XOR again with same key)
decrypted = ''
for char in ciphertext:
    decrypted += chr(ord(char) ^ ord(key))

print("Ciphertext:", ciphertext)
print("Decrypted text:", decrypted)
```

Output:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

sers\hp\.vscode\extensions\ms-python.debugpy-2025.14.1-win32-x64\bundled\libs\debug
sers\hp\Desktop\python workspace\lab_mid_is.py'
Enter message: I am on my Way
Enter single-character key: D
d%)d+*d)=d!!%=
Decrypted text: I am on my Way
PS C:\Users\hp\Desktop\python workspace> █
```

Question # 03:

Write a Python program to decrypt a ciphertext using the Vigenère Cipher. Ask the user for ciphertext and key, and display the decrypted plaintext.

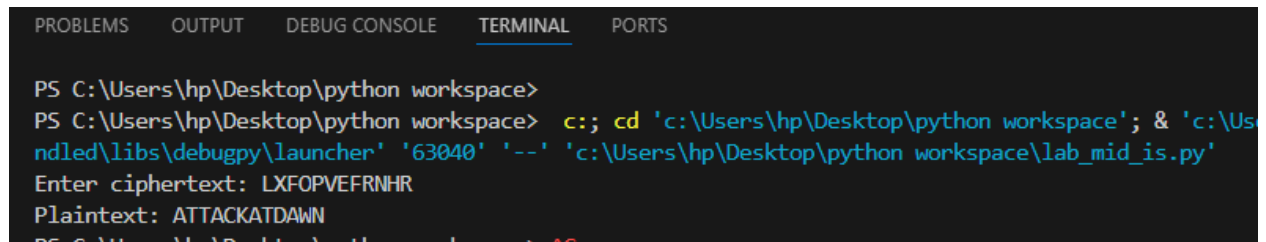
```
ciphertext = input("Enter ciphertext: ").upper()
key = input("Enter key: ").upper()

plaintext = ''
key_index = 0

for char in ciphertext:
    if char.isalpha():
        shift = ord(key[key_index % len(key)]) - ord('A')
        decrypted_char = chr((ord(char) - ord('A') - shift) % 26 + ord('A'))
        plaintext += decrypted_char
        key_index += 1
    else:
        plaintext += char

print("Plaintext:", plaintext)
```

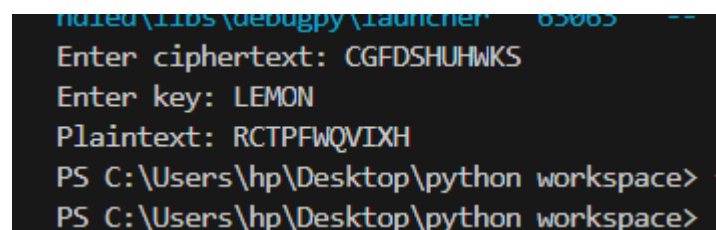
OUTPUT 1:



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

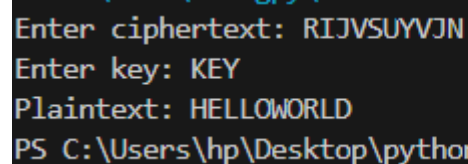
PS C:\Users\hp\Desktop\python workspace>
PS C:\Users\hp\Desktop\python workspace> c:; cd 'c:\Users\hp\Desktop\python workspace'; & 'c:\Users\ndled\libs\debugpy\launcher' '63040' '--' 'c:\Users\hp\Desktop\python workspace\lab_mid_is.py'
Enter ciphertext: LXFOPEFRNHR
Plaintext: ATTACKATDAWN
PS C:\Users\hp\Desktop\python workspace>
```

OUTPUT 2:



```
ndled\libs\debugpy\launcher' '63040' '--'
Enter ciphertext: CGFDSHUHWKS
Enter key: LEMON
Plaintext: RCTPFWQVIXH
PS C:\Users\hp\Desktop\python workspace>
PS C:\Users\hp\Desktop\python workspace>
```

OUTPUT 3:



```
ndled\libs\debugpy\launcher' '63040' '--'
Enter ciphertext: RIJVSUYVJN
Enter key: KEY
Plaintext: HELLOWORLD
PS C:\Users\hp\Desktop\python workspace>
```

Question # 04:

The following program is intended to encrypt text using the Caesar Cipher, but it contains an error. Fix the mistake so that it runs correctly and gives the right output.

- ```
def caesar_encrypt(text, shift):
 result = ""
 for char in text:
 if char.isalpha():
 result += chr(ord(char) + shift)
 else:
 result += char
 return result

msg = input("Enter message: ")
s = int(input("Enter shift: "))
print("Ciphertext:", caesar_encrypt(msg, s))
```

#### Incorrect part in code:

```
result += chr(ord(char) + shift)
```

- **Correct code:**

```
def caesar_encrypt(text, shift):
 result = ""
 for char in text:
 if char.isalpha():
 base = ord('A') if char.isupper() else ord('a')
 result += chr((ord(char) - base + shift) % 26 + base)
 else:
 result += char

 return result

msg = input("Enter message: ")
s = int(input("Enter shift: "))
print("Ciphertext:", caesar_encrypt(msg, s))
```

## OUTPUT:

```
ndled\libs\debugpy\launcher' '63231' '--' 'c:\Users\hp\Desktop\
Enter message: SYSTEM HAS SUCCESSFULLY HACKED.
Enter shift: 3
Ciphertext: VBVWHP KDV VXFFHVIXOB KDFNHG.
PS C:\Users\hp\Desktop\python workspace>
```

## Question # 05:

(a) One similarity between DES and AES:

They are only similar based on the use of **symmetric key**, meaning the same key is used for encryption and decryption. Both are known as symmetric key block ciphers.

(b) CBC mode stands for:

**Cipher Block Chaining** — each plaintext block is XORed with the previous ciphertext block before encryption.

(c) Why is AES faster than DES?

AES uses :

- **fewer, more efficient operations** (based on byte-level substitution and permutation)
- supports **hardware optimization**,

making it faster than DES.